

Introduction:

Road surface anomalies including potholes, surface/external debris and surface damage are a safety hazard for vehicles, pedestrians and eventually infrastructure. Conventional inspection procedures rely more on manual surveys or vehicles mounted sensors which can be expensive, inflexible and a hassle for drivers and commuters. Vision-based approaches exploiting dashcam footage show the potential to provide a scalable alternative, especially in combination with edge AI techniques that enable inference to be locally performed and not dependent on a network.

Edge deployment, however, imposes rigid limitations on compute, power and memory. The Raspberry Pi platform is a widely available Arm-based system which can be used to evaluate such constraints. This project is a look into the feasibility of deploying a real-time object detection pipeline on a Raspberry Pi using CPU-only inference with an eye towards optimizing the system but not on optimizing the model.

Problem Statement and Objectives:

The goal of this project is to create an edge AI application that would be able to detect anomalies on the road based on a dashcam footage in real-time using a Raspberry Pi.

The key objectives are:

1. Allowing real time inference via execution by CPU only
2. Achieving minimum 5 FPS throughput
3. Allows high detection precision and minimize false positive
4. Ensuring Robustness in Changing Lighting Conditions
5. Logging in detected anomalies (including timestamps) to analysis later

System Architecture:

The system contains a four-stage processing pipeline:

1. Input Acquisition - Video frames are captured from a USB webcam or a raspberry pi camera module.
2. Pre-processing - Frames are scaled down and normalized with OpenCV to the input size of the model.
3. Inference - A quantized YOLOv5s model is used for object detection using ONNX Runtime on the CPU.
4. Post-processing & Logging - Bounding boxes are drawn for detected anomalies and timestamped logs are saved.

Hardware and Software Setup:

A. Hardware

- Raspberry Pi 4 - Quad-core Cortex 4 Arm CPU
- Raspberry Pi Camera Module v2
- Power Supply
- USB Pen drive

B. Software

- Raspberry Pi OS (64-bit Linux)
- Python 3
- OpenCV (Good for video processing) # Open CV is one of the earliest tools used for video processing.
- ONNX (Inference measurements) - ONNX Runtime
- YOLOv5(Object detections and analysis)
- Angry IP Scanner (scanning IP address of the connected networks)
- VNC viewer (for virtual pi desktop)

Dataset and Model Selection:

Multiple publicly available data sets were used, from Kaggle. Multi-country data was used to aid in the generalization to different road textures and lighting conditions.

The model taken is the YOLOv5s model which is a compromise between accuracy and computation power. To simplify the inference and to increase the speed, all the road damage classes were combined into two categories:

- 1.Pothole
- 2.Road Damage

Model Optimization:

Running the original floating-point model (FP32) on the Raspberry Pi gave insufficient performance to do this in real-time. To solve this, using quantization of INT8 was applied with ONNX Runtime quantization tools.

Quantization allowed reduction in model size and greatly reduced computational complexity to the point where it is now usable on a CPU. Arm optimized inference kernels and multi-threading were also employed to improve the throughput.

Performance Comparison

Model Format	FPS
FP32 ONNX	~2 FPS
INT8 ONNX	~0.8

Video Pipeline and Logging Using OpenCV:

Frames had to be trapped at a resolution determined by a trade-off between visual detail and CPU load. OpenCV was used for resizing the frame, normalizing and for the annotation.

Whenever an anomaly was observed, bounding boxes along with confidence scores are overlaid on the frame. All detected events were recorded with timestamps and select frames or short clips sent and saved for storage to verify the occurrences.

Performance Evaluation:

The system was tested in a range of lighting conditions, such as during daylight as well as under partial shadows.

Frame rate, CPU usage, and detection accuracy were recorded while the system was running continuously.

The pipeline was found to regularly hit over 5 FPS without thermal throttling once active cooling was used. False Positives in Logging Were Lower Using Precision-Focused Thresholding.

Results and Discussion:

The results have been found to show that real-time road anomaly detection is possible on a Raspberry Pi using CPU inference if the proper optimization techniques are used. While some loss of accuracy was observed from quantization, the trade-off was acceptable since substantial performance improvements were recorded.

Limitations and Future Work:

Heating issue is there in the Raspberry pi Board so to overcome this we'll use fan inbuilt to increase the performance. And FPS is also not good we got so we will try to achieve at least better FPS to comes this tech into real works. Some of the current limitations include poor depth perception and a dependence on only 2D vision. Future work may be able to investigate sensor fusion, adaptive thresholding, or running on higher-performance ARM platforms. Additional

training on datasets that are region specific could also contribute to the enhanced accuracy.